

Problem Solving with AI

(Claude Code + JavaScript)

Antara Aditya

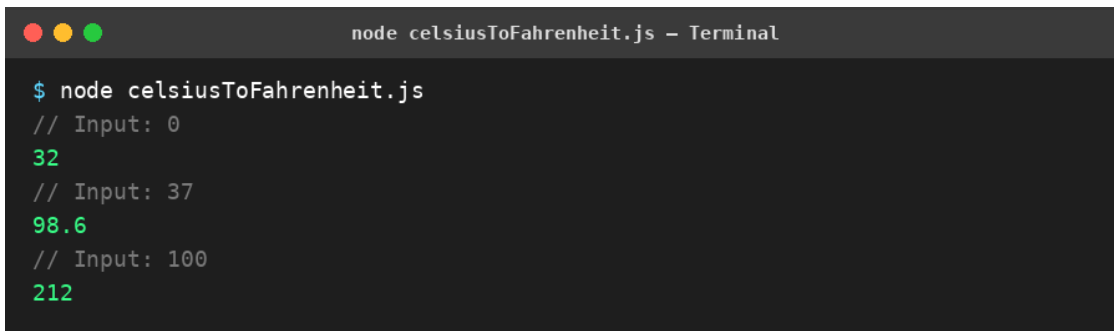
1. Convert Celsius to Fahrenheit

Task: Write a function that takes a temperature in Celsius and converts it to Fahrenheit using the formula: $F = (C \times 9/5) + 32$.

Source Code

```
function celsiusToFahrenheit(celsius) {  
  return (celsius * 9 / 5) + 32;  
}  
  
// Example calls  
console.log(celsiusToFahrenheit(0));    // 32  
console.log(celsiusToFahrenheit(37));   // 98.6  
console.log(celsiusToFahrenheit(100));  // 212
```

Input & Output



```
node celsiusToFahrenheit.js - Terminal  
  
$ node celsiusToFahrenheit.js  
// Input: 0  
32  
// Input: 37  
98.6  
// Input: 100  
212
```

Explanation

- The function `celsiusToFahrenheit(celsius)` takes one parameter, the Celsius value.
- It applies the standard conversion formula $F = (C \times 9/5) + 32$ and returns the result.
- Multiplying by $9/5$ (instead of 1.8) keeps the calculation exact using JavaScript's floating point arithmetic.
- Tested with 0°C (freezing point $\rightarrow 32^{\circ}\text{F}$), 37°C (body temperature $\rightarrow 98.6^{\circ}\text{F}$) and 100°C (boiling point $\rightarrow 212^{\circ}\text{F}$), all matching known real-world values.

2. Find the Factorial of a Number

Task: Write a function that calculates the factorial of a given non-negative integer (e.g., $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$).

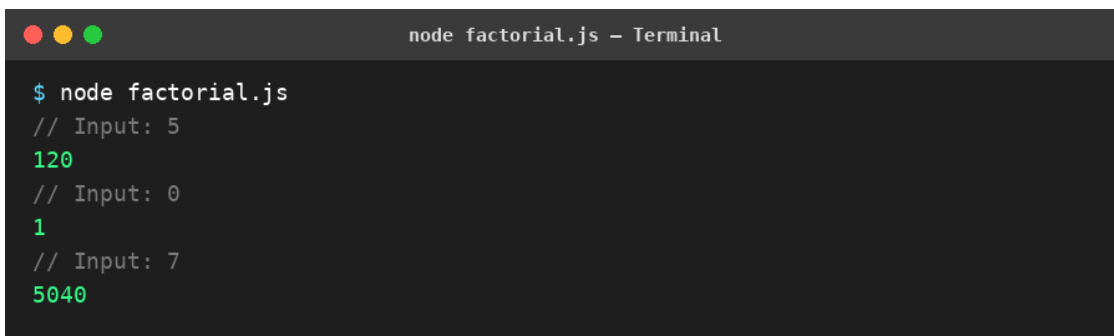
Source Code

```
function factorial(n) {
  if (n < 0) return undefined; // factorial is not defined for negative numbers
  if (n === 0 || n === 1) return 1;

  let result = 1;
  for (let i = 2; i <= n; i++) {
    result *= i;
  }
  return result;
}

// Example calls
console.log(factorial(5)); // 120
console.log(factorial(0)); // 1
console.log(factorial(7)); // 5040
```

Input & Output

A terminal window titled "node factorial.js - Terminal" with three colored window control buttons (red, yellow, green) in the top-left corner. The terminal shows the command "\$ node factorial.js" and its output. The output consists of three lines: "// Input: 5" followed by "120", "// Input: 0" followed by "1", and "// Input: 7" followed by "5040". The input lines are in a light gray font, and the output values are in a bright green font.

```
node factorial.js - Terminal

$ node factorial.js
// Input: 5
120
// Input: 0
1
// Input: 7
5040
```

Explanation

- A guard clause returns `undefined` for negative input, since factorial is undefined there.
- $0!$ and $1!$ are handled as base cases and directly return 1.
- For any other n , a `for` loop multiplies `result` by every integer from 2 up to n .
- This iterative approach avoids recursion overhead and works correctly for reasonably large values of n .

3. Check for Palindrome

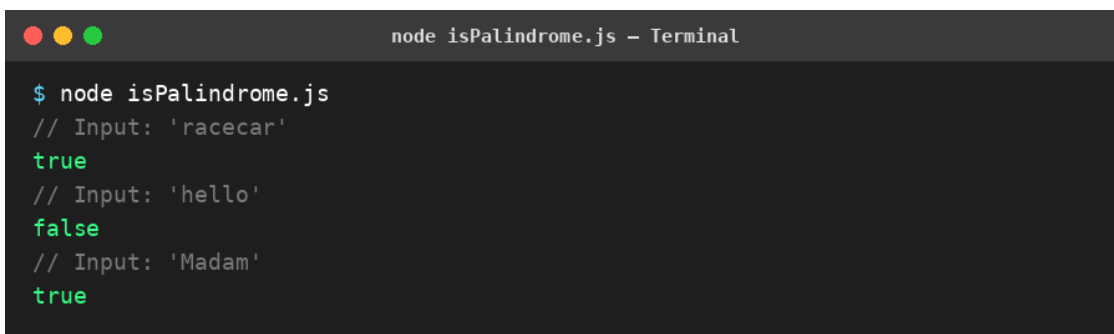
Task: Write a function that checks if a given word (e.g., "racecar") reads the same backward as forward and returns true or false.

Source Code

```
function isPalindrome(word) {
  const cleaned = word.toLowerCase().replace(/^[^a-z0-9]/g, "");
  const reversed = cleaned.split("").reverse().join("");
  return cleaned === reversed;
}

// Example calls
console.log(isPalindrome("racecar")); // true
console.log(isPalindrome("hello"));  // false
console.log(isPalindrome("Madam"));  // true
```

Input & Output



```
node isPalindrome.js - Terminal

$ node isPalindrome.js
// Input: 'racecar'
true
// Input: 'hello'
false
// Input: 'Madam'
true
```

Explanation

- The word is first lower-cased and stripped of any non-alphanumeric characters, so casing and punctuation don't affect the check.
- `split("")` turns the cleaned string into an array of characters, `reverse()` flips their order, and `join("")` rebuilds the string.
- The original cleaned string is compared to the reversed string; if they match, the word is a palindrome.
- "Madam" correctly returns true because the case-insensitive cleaning makes it equivalent to "madam".

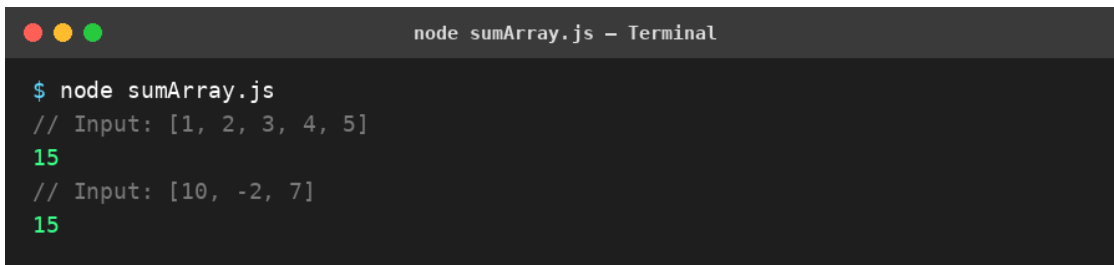
4. Sum of Array Elements

Task: Write a function that takes an array of numbers and returns the sum of all elements in the array.

Source Code

```
function sumArray(arr) {  
  return arr.reduce((total, num) => total + num, 0);  
}  
  
// Example calls  
console.log(sumArray([1, 2, 3, 4, 5])); // 15  
console.log(sumArray([10, -2, 7]));    // 15
```

Input & Output



```
node sumArray.js - Terminal  
  
$ node sumArray.js  
// Input: [1, 2, 3, 4, 5]  
15  
// Input: [10, -2, 7]  
15
```

Explanation

- `Array.prototype.reduce()` walks through every element of the array, accumulating a running total.
- The accumulator `total` starts at 0 (the second argument to `reduce`) and adds each `num` in turn.
- This works for any array of numbers, including negative numbers, as shown in the second example (`[10, -2, 7]` → 15).
- It is a concise, one-line alternative to writing a manual for-loop with an accumulator variable.

5. FizzBuzz (Basic)

Task: Write a loop from 1 to 15 that prints "Fizz" for numbers divisible by 3, "Buzz" for numbers divisible by 5, "FizzBuzz" for numbers divisible by both, and the number itself otherwise.

Source Code

```
function fizzBuzz() {  
  for (let i = 1; i <= 15; i++) {  
    if (i % 3 === 0 && i % 5 === 0) {  
      console.log("FizzBuzz");  
    } else if (i % 3 === 0) {  
      console.log("Fizz");  
    } else if (i % 5 === 0) {  
      console.log("Buzz");  
    } else {  
      console.log(i);  
    }  
  }  
}  
  
fizzBuzz();
```

Input & Output



```
node fizzBuzz.js - Terminal  
  
$ node fizzBuzz.js  
// Input: loop 1 to 15  
1  
2  
Fizz  
4  
Buzz  
Fizz  
7  
8  
Fizz  
Buzz  
11  
Fizz  
13  
14  
FizzBuzz
```

Explanation

- A `for` loop runs from `i = 1` through `i = 15`.
- The condition `i % 3 === 0 && i % 5 === 0` is checked first, since numbers divisible by both 3 and 5 (i.e. 15) must print "FizzBuzz", not "Fizz" or "Buzz" alone.
- If only divisible by 3, "Fizz" is printed; if only divisible by 5, "Buzz" is printed.
- If neither condition is true, the loop simply prints the number `i` itself.